Web Scraping, Vector Embeddings, and Semantic Retrieval Using ChromaDB, LangChain, Ollama, and OpenWebUI

Darious Brown

Walsh College

MAY 2026

Introduction

The purpose of this project was to develop an end-to-end artificial intelligence web scraping and semantic retrieval pipeline using modern retrieval-augmented generation (RAG) components. The solution integrates web scraping, document chunking, vector embeddings, vector database storage, local large language models (LLMs), and semantic search interfaces. The project leverages LangChain for document processing, ChromaDB for vector database management, SentenceTransformers for embedding generation, Docker for containerized deployment, Ollama for local LLM execution, and OpenWebUI for interactive semantic retrieval testing.

Environment Configuration and Architecture

The project environment was constructed using a locally hosted architecture to preserve privacy and enable offline semantic retrieval workflows. Core technologies included LangChain, ChromaDB, SentenceTransformers, Docker, Ollama, OpenWebUI, and Python/Jupyter Notebook. The embedding model used was all-MiniLM-L6-v2 and the local language model used for semantic interaction was llama3.2.

Data Collection and Web Scraping Pipeline

The solution implemented a web scraping pipeline that collected publicly available content from multiple web resources, including sources such as Wikipedia, Reuters, and Microsoft documentation. LangChain's WebBaseLoader extracted webpage content, and CharacterTextSplitter divided content into manageable chunks using approximately 1000-character chunk sizes with overlap to preserve context.

Embedding Generation and Vector Storage

After preprocessing, semantic embeddings were generated using SentenceTransformers. Each document chunk was converted into numerical embeddings suitable for semantic similarity operations. Metadata included source URLs, titles, timestamps, and document characteristics. The completed pipeline successfully processed 276 documents with 384-dimensional embeddings stored inside a ChromaDB collection.

Semantic Retrieval Testing

Semantic retrieval testing was conducted using OpenWebUI connected to Ollama. Natural language prompts were used to validate contextual understanding and semantic retrieval. The system successfully produced context-aware responses based on the embedded document corpus, confirming proper vector similarity search and local LLM integration.

Challenges and Troubleshooting

Implementation challenges included dependency conflicts involving HuggingFace libraries, transformers, and TensorFlow/Keras compatibility. Additional troubleshooting involved Docker container validation, ChromaDB heartbeat testing, environment variable configuration, notebook corrections, and repository synchronization.

Conclusion

This project successfully implemented an AI-powered semantic retrieval pipeline integrating web scraping, preprocessing, embeddings, vector storage, and semantic retrieval. The project demonstrates practical Retrieval-Augmented Generation concepts and highlights the importance of vector databases, embeddings, and local LLM ecosystems within modern AI workflows.